

CPT202 Assignment 2
Group Report for Software Engineering Group Project

2024/2025 Semester 2

An online meeting booking system

April 30 2025

<http://114.55.137.200>

Group number: 33

Group Members:

1. <Lekun Liu , Student ID 2255714>
2. <Yiyang Han , Student ID 2252155>
3. <Junhao Huang , Student ID 2256792>
4. <Yichuan Zhang , Student ID 2251668>
5. <Tiantian Ye , Student ID 2251663>
6. <Daiyan Wu , Student ID 2257387>
7. <Ruiyang Wu , Student ID 2257475>
8. <WeiZhe Sun , Student ID 1927150>
9. <Xuning Hu , Student ID 2251992>

Contents

Introduction	2
Background and Requirements	2
Aims of The Project	2
User Characteristics	2
Assumptions and Dependencies	2
Project Scope	3
Project Risks	3
Architectural Design	3
Design Requirements	3
System Architecture	4
Software Design	6
Software Modules	6
High-level Process Flow	7
Software Support Services	10
Coding Structure and Convention	10
Software Configuration and Production Environment	11
Software Testing	11
Unit Testing	11
Integration Testing	12
Acceptance testing	12
Conclusion and Future Work	13
References	14
Appendix	15

Introduction

Background and Requirements

Universities have traditionally relied on paper-based/manual systems for managing meeting room bookings, resulting in scheduling conflicts, underutilized spaces, and lack of real-time visibility into room status. To address these pain points, our project delivers a centralized, web-based platform that brings real-time transparency and efficiency to campus meeting room management system.

The system must accommodate two primary stakeholder groups. Administrators need a consolidated view of all rooms, users and bookings, so they can quickly adjust room details or timely control and management of user accounts and room availability. Students require a straightforward, self-service interface for account creation, profile updates, and reservation management. These core requirements underpin a solution that streamlines booking workflows, reduces administrative overhead, and enhances the user experience.

Aims of The Project

The goal is to deliver a seamless reservation system tailored for students and administrators. Students can browse real-time room availability, filter by capacity, location, or equipment, and reserve or edit bookings with ease. Administrators can manage room inventories and handle reservations, including suspending users or approving special requests. The system follows security best practices, including email verification, password hashing, and secure session handling. Visual feedback is provided throughout to ensure both user autonomy and administrative oversight.

User Characteristics

1 System Administrator: Resource curator

Full-time IT or facilities staff who use the system daily to manage requests, update room data (e.g., post-renovation), and monitor feedback. They need efficient tools to approve or reject bookings and manage accounts.

2 End user: University student

Undergraduate or postgraduate students who prefer quick, mobile-friendly booking. They value intuitive filters, real-time availability, and flexible management. They expect autonomy, schedule alignment, and detailed room previews.

Assumptions and Dependencies

User Assumptions

The system assumes that all end users which are mainly students, possess stable internet connectivity, a device compatible with modern web browsers, and a valid university email address for registration. Ensures access to email verification and real-time booking.

Technical Dependencies

The back-end uses Spring Boot, MySQL and MyBatis-Plus (for streamlining database interactions). The front-end is built with Vue framework alongside standard HTML/ CSS/ JavaScript for responsive UI rendering. Key services include SMTP for email, Caffeine for caching, and Qiniu for object storage. We used tools like Hutool (utility libraries) and Knife4j (Swagger-based API documentation) to aid development. Deployment is on Alibaba Cloud server, ensuring scalability and reliability for institutional use cases.

Project Scope

The scope of the project includes the following:

- Students access a registration page where they verify their email, set a secure password, and optionally upload an avatar. Once logged in, they can navigate to their profile page to update personal details, change passwords, or replace their avatar.
- The rooms page displays all available meeting rooms in list format with room name, type and location. Users can apply filters, such as capacity, location, or facilities. Users can click on a row to view specific room information or book it.
- A booking record page to view historical bookings and reservations pending confirmation. Users can reset or cancel the meeting time that have not yet started here.
- Administrator use the Room Management page to create a new room or update the list of existing rooms to define capacity, device sets, location details, and upload image. They can also deactivate rooms to ensure catalogs are to date and accurate. A user management page for locking, unlocking, or deleting users.
- The Reservation Management page lists all user reservation requests, allows, rejects or flags them for further review, and responds to user requests for information.
- A dedicated section captures student feedback on room experiences.

Project Risks

- **Integration Complexity:** Dependency on Spring Boot, MyBatis-Plus, SMTP, Caffeine, Qiniu Cloud, and MySQL raises the risk of compatibility or Configuration issues.
- **Usability Challenges:** Poor UI/UX in filters, calendar views, or email steps may reduce user adoption.
- **Data Breach & Unauthorized Access:** Weak session control or password hashing may compromise sensitive data.
- **Injection Vulnerabilities:** Unprotected SMTP endpoints could be exploited for spamming if no request limits are set.
- **Ethical Considerations:** Non-compliance with university or local data laws could lead to legal risks regarding student data handling.

Architectural Design

Design Requirements

Key requirements shaping our architecture include real-time availability tracking, secure authentication with role-based access control and email verification, modular separation of concerns for maintainability and testability, RESTful API compliance, fault clarification via logs, and cloud-based deployment and nginx reverse proxy for elastic and high efficient loading operations. The overall code structure follows the formatting design for eventual code testing [1].

System Architecture

The system follows a modular, layered architecture: a Vue.js front end and Spring Boot back end communicate over RESTful APIs, with MyBatis-Plus managing MySQL persistence and Qiniu Cloud for media storage. Code versioning on GitHub supports collaborative development, and deployment is handled via Nginx with load balancing, ensuring future scalability and high availability.

The **front-end** configures a proxy to forward API requests with the `/api` prefix to the local Spring Boot service (port 8080), effectively solving cross-origin issues during development. After system deployment, Nginx listens on port 80, hosts the compiled front-end static resources, and continues to proxy all `/api` requests to the back-end service, ensuring users access front-end and back-end resources through a unified entry point. The current deployment structure is a single instance deployment, without load balancing mechanisms yet.

The **back-end** service uses MyBatis-Plus to manage persistent interactions with the MySQL database, simplifying database operations and improving development efficiency. For multimedia content in meetings, the system integrates Qiniu Cloud Storage service to uniformly manage uploads and access, further reducing the load on the local server. The project code is hosted on GitHub, supporting team collaborative development and version control, ensuring standardization and traceability in the development process.

Figure.1 illustrates the overall project structure and code versioning flow, highlighting how modularity and standardization are maintained across the codebase. Meanwhile, Figure.2 demonstrates the current system deployment architecture, where front-end and back-end services are decoupled but unified under a single access point managed by Nginx.

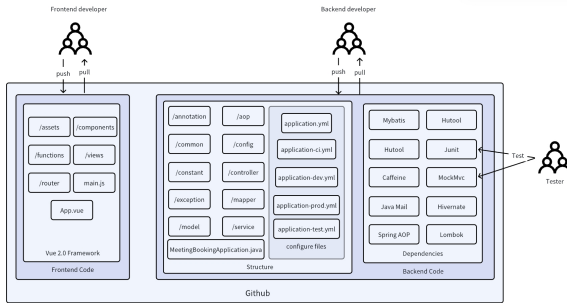


Figure 1: Project Coding Structure and Version Control

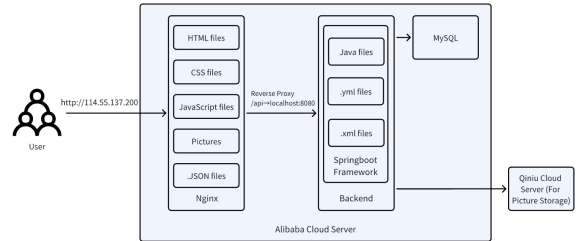


Figure 2: Project Deployment and Brokering

The overall architecture implements a system operation mode with clear front-end and back-end responsibilities and efficient communication, supporting both independent deployment and debugging, while also providing a solid architectural foundation for future system expansion and functional evolution.

Front-end System

From implementation-oriented view, our front-end is a component-driven Application built with Vue 2 framework. The project adopts a hybrid structure combining functional layering with modular business separation, which ensures a clear separation of concerns, promotes code reusability, and keeps business logic well-organized and scalable. The application is bootstrapped with vue-cli for rapid project scaffolding, and relies on vue-router for routing and page navigation [2]; global state is managed via Vuex, storing user information and permission identifiers. The network layer uses Axios to centralize API calls. We also have our front-end system to automatically attach the back-end-issued

session ID in request interceptors and store it in a cookie to persist user identity. Cross-cutting concerns such as global loading indicators, error notifications, and permission checks are implemented through Axios interceptors. Code quality is enforced by ESLint + Prettier. The build process employs vue-cli's built-in Webpack configuration for code splitting, and static assets are ultimately published via the Vue build output.

From UX architecture view, we used Figma to design the page prototypes, aiming to meet various user experience requirements such as clear information hierarchy and consistent interaction patterns. The prototypes focus on providing intuitive navigation, predictable user flows, and visual clarity through well-defined layouts and component behavior. We structured the application into clear, function-based modules, resulting in four main pages for regular users: Rooms, My Bookings, Feedback, and Profile. For administrators, two additional pages—Users and Booking Review—are accessible. Each page is mapped to a distinct route and corresponds to a specific user task or role, ensuring a clean navigation experience and clear functional separation. In addition, we ensured a consistent and predictable user experience by standardizing page layouts, UI component interaction patterns, functional button styles and behaviors, as well as feedback mechanisms across the entire application. This design consistency reduces the learning curve for users and enhances overall usability. We then customized necessary UI components and style modules on Vue, leveraging scoped CSS and BEM conventions to ensure high reusability and style isolation.

Back-end System

Our back-end is built as a modular, service-oriented system using Spring Boot at its core to rapidly scaffold RESTful APIs, manage dependencies, and leverage the extensive Spring ecosystem for security, transaction management, and configuration [3]. We chose MyBatis-Plus atop MySQL to combine the stability and maturity of a relational database with powerful, type-safe ORM capabilities. Its 'QueryWrapper' could be used to build dynamic SQL in Java without boilerplate. Caffeine Cache addresses performance and scalability by caching ephemeral data like email verification codes with automatic eviction policies, while the Hutool utility suite accelerates common tasks like string validation and bean copying. All user-uploaded assets are offloaded to Qiniu Cloud Object Storage, reducing server disk usage and exploiting CDN acceleration to deliver images quickly to users. Email workflows integrate with NetEase 163's SMTP service for reliable delivery of verification codes. API documentation is auto-generated via Knife4j (Swagger), ensuring that developers always have interactive reference material. Cross-cutting concerns, such as logging, exception handling, and secure execution, are implemented through Spring AOP [4] and custom annotations, with identity authentication based on role-based access control (RBAC). This combination of proven frameworks and utilities ensures rapid development, high efficiency in objects management, strong security, and easy future extension.

Subsequent sections provide technical details on the front-end, back-end, and database implementations, explaining how these components work together to meet system requirements and performance goals.

High-level Database Design

Our system schema follows principles of normalization and referential integrity. As shown in Figure.3, the core entities are:

- **Users:** The users table uses `user_id` as its primary key to uniquely identify each account. It contains fields for username, email, password, role, and audit timestamps. As the

cornerstone of the system, users is referenced by bookings via user_id and feedback via user_id and admin_id, enforcing referential integrity for all actions initiated or approved by a user.

- **Buildings:** With building_id as its primary key, the buildings table catalogs each physical location. It includes a unique name, and timestamps. By centralizing building data, the design enables every room record to reference its site through rooms.building_id, simplifying campus-wide room grouping and future expansion without redundant location entries.
- **Room_types:** The room_types table's primary key is type_id, This table standardizes classifications such as "Class Meeting Room" or "Conference Room." Each room record points to a type via rooms.type_id, and the same type of room shares the same capacity limit.
- **Rooms:** rooms uses room_id as its primary key and links to both buildings.building_id and room_types.type_id. Additional fields capture capacity, facilities, and image references. This dual foreign-key structure tightly couples rooms to their physical and categorical contexts, enabling filtered searches and coherent management.
- **Bookings:** The bookings table's primary key is booking_id, which anchors each reservation. Foreign keys user_id and room_id tie bookings to the requested student and targeted room, while reviewer_id links to an administrative user for approval workflows. This schema supports record trails, status tracking, and conflict prevention through enforced relationships.
- **Feedback:** The primary key of Feedback table is feedback_id and it also contains user_id and admin_id, both as foreign keys into users table. This dual reference permits both sides clearly identify who the respondent and the sender are respectively.

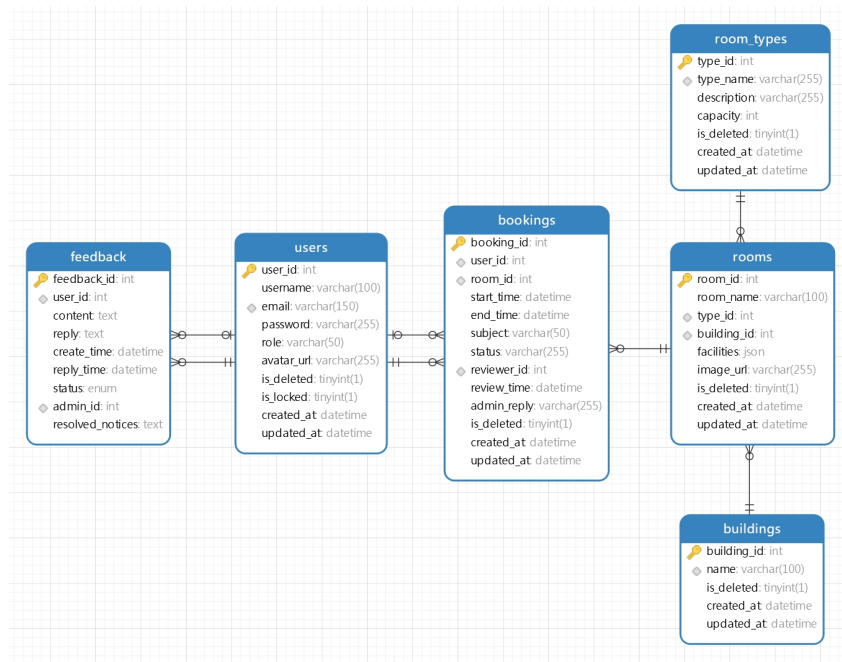


Figure 3: ER diagram

Software Design

Software Modules

The system architecture comprises five core modules: Authentication Module, User Management Module, Booking Management Module, Room Management Module, and Feedback Module. Authentication handles secure login and session control; User Management controls administrator and user entity operations; Booking Management coordinates reservation workflows; Room Management maintains room data; Feedback captures and processes user feedback.

To illustrate how different roles interact with the system’s core modules, we present two activity diagrams—one for regular users (Figure.4) and one for administrators (Figure.5). These diagrams concisely depict the typical workflows and module interactions for each role.

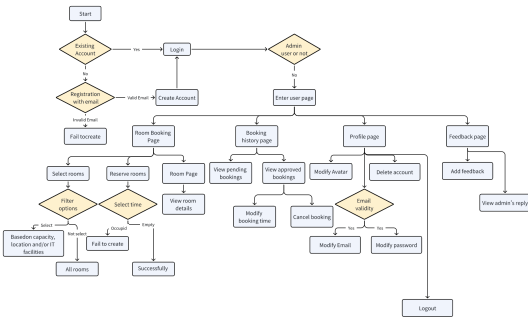


Figure 4: User Activity Diagram

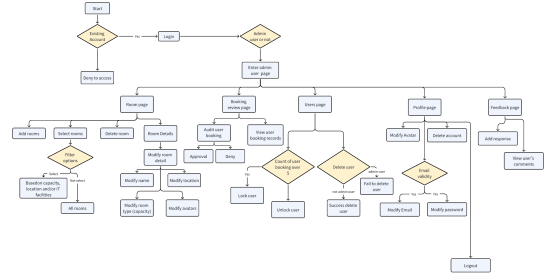


Figure 5: Admin Activity Diagram

High-level Process Flow

Authentication Module

The Authentication Module is the core security component of the system, responsible for user identity verification, session management, and security enforcement. It integrates email-based verification, password encryption, role-based access control.

- **User registration:** The `/user/register` interface processes registration requests through `UserRegisterRequest` object which including username, password, email, and optional avatar by using POST method. The back-end verifies email Captchas via `verifyEmailCode` method, encrypts passwords with MD5 + Salt (“group33”), and uploads avatars to Qiniu cloud only after successful registration to prevent dirty data. Successful registrations return desensitized `UserVO` data.
- **User login:** The `/user/login` interface accepts username or email credentials within `UserLoginRequest` object by POST method, verifies encrypted passwords against database records, and establishes `HttpSession` sessions. User sessions are maintained under `USER_LOGIN_STATE` with sensitive data hidden through `getLoginUserVO` desensitization.

User management module

The User management module consists of two sub-modules: the admin user management module and the user information control module. This module handles all the relevant

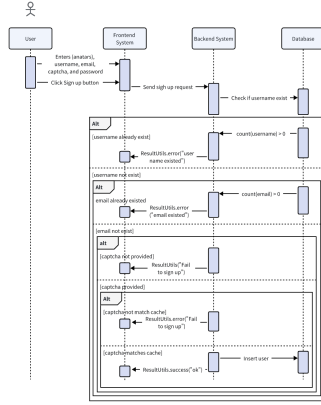


Figure 6: Sign Up Sequence Diagram

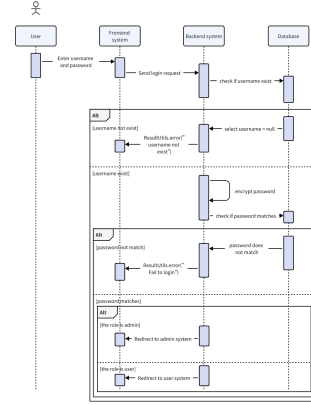


Figure 7: Login Sequence Diagram

operations, including CRUD operations for user information and user management functions for system administrators. These two modules have different permission restrictions for administrators and normal users.

- **Admin user management:**

The admin user management module handles the requirements of adding and deleting users, locking and unlocking users, and viewing users' detailed information. This module has multiple endpoints under `/admin/users`. To view all users' detailed information, administrators can get the user list through `/admin/users/list`. The `/admin/users/lock/{userId}` and `/admin/users/unlock/{userId}` endpoints are used to lock and unlock overbooking users. For deleting existing users, the `/admin/users/delete/{userId}` is used to delete the specific user. For administrators to add a new user, `/admin/users/add` is a POST method endpoint to add a new user with needed user informations from the administrator page.

- **User information management:**

The user information module handles the requirements of updating and retrieving users' personal information. This module has multiple endpoints under `/user`, for updating a user's information such as changing username, email and password, uploading and deleting avatar. These endpoints are designed as POST methods for users to upload the required information.

Room management module

The Room Management Module manages meeting room data (create, modify, delete, retrieve) via role-based access control (RBAC). All users can view room details, but only administrators can perform write operations (create, update, delete). Room images are stored securely on Qiniu Cloud. Main functions are detailed below.

- **Room viewing:** The `/room/list`, `/room/searchById`, and `/room/detail` endpoints allow both regular users and administrators to access room information. Users can browse all available rooms, search by ID, and view detailed metadata, such as capacity, available facilities, time slots, and associated images. The result will be placed in RoomDetailVO and transmitted back to the front-end.
- **Room management:** Administrators can manage room resources through `/admin/room/add`, `/admin/room/update`, and `/admin/room/delete` endpoints. These interfaces support full CRUD operations, allowing administrators to add new rooms with

metadata and image uploads, edit existing room information, or delete obsolete rooms. Uploaded images are stored in Qiniu Cloud, and the system ensures data consistency between the image storage and database records.

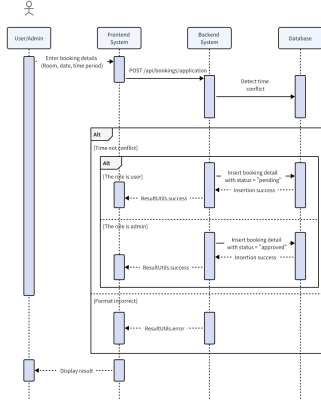


Figure 8: Create New Booking Sequence Diagram

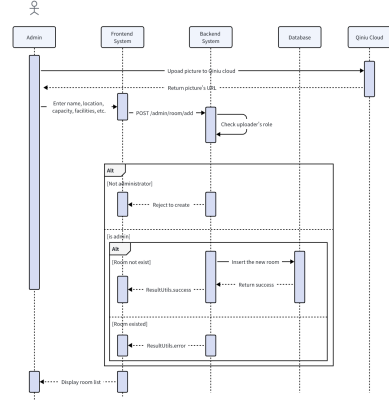


Figure 9: Create New Room Sequence Diagram

Booking management module

The booking management module is designed for full lifecycle management of meeting room reservations. This interface architecture is built on the Spring framework's Controller design pattern, providing standardized processing of HTTP requests from clients and returning structured response data.

- Booking Information Retrieval:** The system provides multiple endpoints for accessing booking information to meet various user needs. Users can retrieve their upcoming reservations through `/bookings/list` with GET method, which returns coming bookings in BookingListVO format with essential reservation details. Historical booking records are accessible via `/bookings/history` by GET method, which employs time comparison algorithms to retrieve completed reservations. For comprehensive information about specific bookings, users can access `/bookings/{booking_id}` via GET method, which returns complete booking information in BookingDetailVO format with appropriate role-based access controls.
- Booking Creation and Modification:** The module supports the full creation and modification lifecycle of booking records. New reservations are created through the `/bookings/application` by POST method, which accepts BookingApplicationRequest objects containing all necessary reservation parameters and implements conflict detection. Existing bookings can be modified via `/bookings/{booking_id}/modify`, which features differentiated permission controls based on user roles and automatically resets modified bookings to a pending review status when changed by regular users.
- Booking Status Management:** The system provides functionality for managing booking statuses throughout their lifecycle. The cancellation endpoint `/bookings/{booking_id}/cancel` allows users to cancel existing reservations, implementing dual verification through both role-based and ownership-based permission checks before changing the booking status to cancelled.

Feedback module

The module supports the feedback submission and review lifecycle between users and administrators.

- **User Actions: Submission and Viewing**

New feedback is submitted by users through the `/feedback/add` endpoint using POST method, which the corresponding method in the `FeedbackController` class accepts `FeedbackCreateRequest` objects containing the feedback content and links it to the logged-in user by local session, setting the initial status to pending. Users can retrieve their own feedback lists by `/feedback/allResolved` for resolved items and `/feedback/allPending` for pending items. Detailed information for a specific feedback entry, including any administrative replies, can be accessed using the `/feedback/{feedbackId}` endpoint via GET method.

- **Administrator Actions: Review and Reply**

The service layer also implements functionalities for administrators to reply to feedback, meanwhile updating its status to “resolved”, and to view lists of all pending or resolved feedback across users.

Software Support Services

- **Database Services:** MySQL provides relational data storage, with MyBatis-Plus in Spring Boot simplifying ORM operations. MyBatis-Plus’s `QueryWrapper` simplifies the database query process, reducing SQL boilerplate.
- **Caching & Utility Services:** Caffeine Cache stores short-lived data like 3-minute email codes. Hutool offers utilities for string/file operations and object copying (`BeanUtil.copyProperties`).
- **Security & Email Services:** Authentication uses MD5 + Salt hashing and rate-limited Captcha. NetEase 163 Mail’s SMTP (configured via Spring Boot mail) handles verification emails, with settings in `application.yml`.
- **Cloud Storage & CDN:** Qiniu Cloud stores user avatars/room images, providing CDN acceleration and lifecycle management for cost optimization, with configures managed via Qiniu SDK.
- **Development & Deployment Tooling:** Using IntelliJ IDEA (back-end) and VS Code (front-end) with Git/GitHub to develop collaboratively. CI/CD pipelines to test commits and pull requests.
- **Documentation & Community Support:** Swagger (via Knife4j) documents RESTful APIs. Spring Boot/MyBatis-Plus/Qiniu docs and Stack Overflow/CSDN provide troubleshooting support.

Coding Structure and Convention

Coding structure

Our coding structure adheres to the Model-View-Controller (MVC) pattern [5] [6], emphasizing core principles like separation of concerns, high cohesion with low coupling, maintainability, and testability, while implementing a RESTful API design using resource-oriented URLs and standard HTTP methods. The Controller layer handles incoming HTTP requests and formulates responses, featuring controllers for different modules.

Following this, the Service layer, such as `UserService` and `RoomService`, which encapsulates core business rules and orchestrates workflows, acts as an intermediate layer between controllers and data access. The Model and Data Access layer defines entity objects, such as user and room, mapping to database tables, along with DTOs/VOs for data transfer, like `UserMapper` and `RoomMapper`. Furthermore, there are several supportive packages. For instance, `/common` provides utilities like `ResultUtils` and `EmailSender`; `/config` centralizes Spring Boot configurations; `/constant` defines global values like `UserConstant`; `/exception` standardizes error handling; and `/aop` and `/annotation` implement role-based access control via `AuthInterceptor` and `@AuthCheck`. The `MeetingBookingApplication.java` serves as the application's entry point. This structure promotes modularity, simplifies testing and maintenance, and provides a solid foundation for future development.

For the front-end's code structure, page routing logic is managed by `vue-router`. The user interface code is organized with distinct directories for views which representing web pages and reusable components. All external library dependencies for the front-end project are declared and managed within the `package.json` file.

Coding conventions

We adhere to a unified naming convention throughout the project: `snake_case` is used for all database fields, JSON properties, method names, and variable identifiers—ensuring consistency between Java code, SQL schemas, and RESTful payloads. Class names follow `PascalCase`, aligning with Java standards, while constants in enums use `UPPER_SNAKE_CASE`. Every controller method consumes a dedicated `request_dto` and returns a `response_vo`, enforcing strict contracts for inputs and outputs. Error conditions are all defined as constants and thrown as `BusinessException`, and mapped to uniform API responses. Comprehensive Javadoc, comments, standardized logging statements are based on the Alibaba Java Development Manual, guarantee readability, maintainability, and code quality.

Software Configuration and Production Environment

Software configuration

The back-end configuration utilizes Spring Boot's profile-based YAML files, splitting environments into different files like `dev`, `test`, `prod`, `ci`. These manage core settings and environment-specific details and third-party service integration, ensuring environment consistency. Critical information, such as database connections, is set as environment variables to secure credential separation.

Production environment

Since our project is using the frontend and backend separation architecture, Nginx HTTP web server is used to host the static `Vue.js` as front-end web and reverse proxy the back-end server from internal port to `/api`. The back-end server is listening on `localhost:8080`, which is not exposed externally to reduce unauthorized access risk. This setup shields back-end endpoints from direct exposure, enhancing overall system security. The deployment architecture balances maintainability, performance, and security, adhering to modern web application best practices.

Software Testing

The two tables in the appendix (Table.1 and Table.2) indicate the detailed test data and results, as well as the test techniques used.

Unit Testing

Unit testing focuses on verifying functionality, correctness, and accuracy at the unit level. To avoid real database operations or external dependencies, we used JUnit5 with Mockito to create method stubs, and MockMvc to simulate HTTP requests and validate controller responses. We achieved 100% class and over 80% method test coverage for the Controller and Service (The rest are configuration classes and constant enumeration classes). The following are three essential unit test examples: verify the correctness of the email address (Common), getting the list of users(Service), and getting a Booking based on the Booking ID (Controller).

The unit test in Listing.1 verifies email address correctness (RFC 5322). It tests valid and invalid cases separately by asserting the 'isValidEmail' method results, ensuring the system accurately processes various email formats and improves robustness.

The unit test in Listing.2 validates the admin user service's ability to retrieve the user list. Using Mockito, we mocked dependencies like 'UsersMapper' and 'QiniuConfigure'. We first created a simulated user data object, userVO, and set its property values. Then, we simulated the return value using the when method. We asserted that user ID, username, and avatar URL were correct. In this way, we ensure the correction of service layer logic.

The unit test in Listing.3 checks the controller's functionality to fetch appointment details by ID. MockMvc was used to simulate an HTTP GET request with session info. We mocked the response of 'bookingService', sent a request to /admin/bookings/{bookingId}, and validated the status code and returned booking details, ensuring controller reliability. In this way, we can ensure the controller layer correctly process the request and return the correct response.

Integration Testing

The goal of integration testing is to check the correctness of communication and relationships between different units. To prevent the database from being polluted by tests, we used h2 database instead of the MySQL for testing. Below is an example of an integration test that tests whether a locked user can be unlocked correctly, verifying the interaction between the service layer and the data access layer:

In the example of Listing.4, we used '@SpringBootTest', '@Transactional', and '@ActiveProfiles("test")'. 'AdminUserService' and 'UsersMapper' were injected, while 'EmailSender' was mocked with '@MockBean'. A test user with default information was inserted into the database before each test. We simulated the user being locked, then called 'adminUserService.unlockUser(testUserId)', and asserted via database queries that the user status changed to unlocked. This verifies the interaction between service and data access layers and ensures correct status updates.

Acceptance Testing

The following content is an example of an Acceptance Test, where comprehensive and extensive acceptance tests were conducted to verify that the system functioned in accordance with the design expectations as well as all user requirements. (You may see pictures in Appendix from Figure.16 to Figure.21)

Acceptance Test Case 001: User booking conference room

Test Steps:

1. Login to the system with a test account
2. Navigate to the "Rooms" page
3. Filter available room list
4. Choose one meeting room and set a specific meeting time (tomorrow 19:00–21:00)
5. Fill in the purpose of the booking: *'Team Meeting'*
6. Click on the 'Book' button

Expected Results:

1. The system should display a list of available conference rooms
2. After selecting a meeting room, the details of the meeting room should be displayed correctly
3. User should receive a success message after submitting the booking
4. Booking information should appear in the 'My Bookings' list

Results: The system successfully displayed the list of meeting rooms, and when the Room was selected, its capacity and equipment information was correctly displayed. When submitting the booking, the system verified the time conflict and successfully created the booking record.

Conclusion and Future Work

- **Project Summary and Achievements:** Our group successfully designed, developed, documented, and deployed a comprehensive online meeting room booking system tailored for a university environment. This involved thoroughly defining requirements, aims, user characteristics, scope, and risks. The system features a robust architecture with a Vue.js front-end communicating via RESTful APIs with a Spring Boot back-end, utilizing MySQL managed by MyBatis-Plus, and integrating Qiniu Cloud for media storage. Key functionalities across Authentication, User Management, Room Management, Booking Management, and Feedback modules were implemented for distinct user and admin roles. We established clear coding structures, configured environments, executed a rigorous testing strategy including unit, integration, acceptance, and deployed the project on an Ali Cloud server using Nginx reverse proxy.

Building on this foundation, students can effectively register, manage profiles, filter and browse rooms, check real-time availability, and make bookings, while administrators can efficiently manage rooms, users, bookings, and feedback. Secure user authentication, such as email verification, password hashing is implemented, and the modular architecture ensures maintainability. Key external services like Qiniu Cloud and SMTP were successfully integrated. Our testing achieved substantial coverage, including 100% class and over 80% method coverage in unit tests for critical layers, with core functionalities passing acceptance testing.

- **Development Process:** Our development strictly followed an organized, four-sprint structure adhering to Agile principles, facilitating steady progress [7]. Sprint 1 focused on initial design and setup. Sprint 2 implemented foundational back-end structures and front-end components. Sprint 3 delivered core logic implementation and API integration. Finally, Sprint 4 concentrated on deployment, comprehensive testing, and production verification. This iterative approach allowed us to build and validate the system methodically.
- **Future Enhancements:** Looking ahead, we plan several improvements for continuous enhancement. Key areas include implementing high-concurrency management using rate limiting and potentially message queues, and optimizing performance through multi-level caching strategies (Caffeine, Redis). We also aim to enhance scalability and availability by introducing load balancing beyond the current single-instance deployment. Furthermore, we will address remaining risks, refine security

measures, improve the currently limited front-end styling, incorporate user feedback for UI/UX enhancements, and potentially implement additional features based on future priorities.

References

- [1] F. Zhang, G. Sun, B. Zheng, and L. Dong, “Design and implementation of energy management system based on spring boot framework,” *Information*, vol. 12, no. 11, p. 457, 2021.
- [2] N. Tran, “Applying vue. js framework in developing web applications,” 2020.
- [3] M. Gajewski and W. Zabierowski, “Analysis and comparison of the spring framework and play framework performance, used to create web applications in java,” in *2019 IEEE XVth International Conference on the Perspective Technologies and Methods in MEMS Design (MEMSTECH)*. IEEE, May 2019, pp. 170–173.
- [4] G. Noor and G. Thomas, “Aspect-oriented programming: Transforming code modularity in spring ecosystems,” 2021.
- [5] A. Catrina, “A comparative analysis of spring mvc and spring webflux in modern web development,” 2023.
- [6] J. Zhao, “Application and practice of mvc architecture pattern in on-the-job internship management system,” in *2022 International conference on networks, communications and information technology (CNCIT)*. IEEE, 2022, pp. 25–30.
- [7] B. Grebić and A. Stojanović, “Application of the scrum framework on projects in it sector,” *Eur. Proj. Manag. J*, vol. 11, no. 2, pp. 37–46, 2021.

Appendix (Not count into the limit of 10 pages)

Test Cases Codes

```
1 package com.cpt202g33.meetingbooking.common;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5
6 import static org.junit.jupiter.api.Assertions.*;
7
8 // MushihimePepsi
9 class EmailValidatorTest {
10
11     // MushihimePepsi
12     @Test
13     void testValidEmails() {
14         assertTrue(EmailValidator.isValidEmail("user@example.com"));
15         assertTrue(EmailValidator.isValidEmail("user.name+tag+sorting@example.com"));
16         assertTrue(EmailValidator.isValidEmail("x@example.co.uk"));
17         assertTrue(EmailValidator.isValidEmail("customer/department=shipping@example.com
18     "));
19         assertTrue(EmailValidator.isValidEmail("user_name@example.travel"));
20         assertTrue(EmailValidator.isValidEmail("user@sub-domain.example.com"));
21     }
22
23     // MushihimePepsi
24     @Test
25     void testInvalidEmails() {
26         assertFalse(EmailValidator.isValidEmail(null)); // null
27         assertFalse(EmailValidator.isValidEmail("")); // empty
28         assertFalse(EmailValidator.isValidEmail("plainaddress")); // missing @
29         assertFalse(EmailValidator.isValidEmail("@no-local-part.com"));
30         assertFalse(EmailValidator.isValidEmail("user@.invalid.com"));
31         assertFalse(EmailValidator.isValidEmail("user@invalid..com"));
32         assertFalse(EmailValidator.isValidEmail("user@@example.com"));
33         assertFalse(EmailValidator.isValidEmail("user@ example.com")); // space
34     }
35 }
```

Listing 1: Unit Test Case 1 for Email Verification

```
1 @ExtendWith(MockitoExtension.class)
2 public class AdminUserServiceImplTest {
3
4     @InjectMocks
5     private AdminUserServiceImpl adminUserService;
6
7     @Mock
8     private UsersMapper usersMapper;
9     @Mock
10    private BookingMapper bookingMapper;
11    @Mock
12    private RoomMapper roomMapper; // Assuming the full name based on context
13    @Mock
14    private BookingService bookingService;
15    @Mock
16    private QiniuConfigure qiniuConfigure;
17
18    @BeforeEach
19    void setUp() {
20        MockitoAnnotations.openMocks(this); // Assuming 'this' as the standard argument
21    }
22
23    @Test
24    void testGetUserLists_success() {
25        // Prepare simulated returned user data
26        UserVO userVO = new UserVO();
27        userVO.setUserId(123);
28        userVO.setUsername("testUser");
29    }
```

```

29     userVO.setAvatar_url("avatar/test.png"); // simulated database returns key, not
complete URL
30
31     // mock method actions
32     when(usersMapper.getUserWithBookingCount()).thenReturn(Collections.singletonList
(userVO));
33     when(qiniuConfigure.getDomain()).thenReturn("http://cdn.qiniu.com/"); //
Assuming standard domain structure
34
35     // call adminUserService method
36     List<UserVO> result = adminUserService.getUserLists();
37
38     // Verification results
39     assertEquals(1, result.size());
40     assertEquals(123, result.get(0).getUserId());
41     assertEquals("testUser", result.get(0).getUsername());
42     // Combine domain and key for the final URL assertion
43     assertEquals("http://cdn.qiniu.com/avatar/test.png", result.get(0).getAvatar_url
());
44 }
45 }
46

```

Listing 2: Unit Test Case 2 for Getting User List

```

1  @WebMvcTest(AdminBookingController.class)
2  @EnableAspectJAutoProxy
3  @Import({AuthInterceptor.class}) // Assuming AuthInterceptor class based on context
4  public class AdminBookingControllerTest {
5
6      @Autowired
7      private MockMvc mockMvc;
8
9      @MockBean
10     private BookingService bookingService;
11     @MockBean
12     private UsersService usersService; // Corrected based on convention
13
14     // mock the Mapper
15     @MockBean private com.cpt202g33.meetingbooking.mapper.BookingMapper bookingMapper;
16     @MockBean private com.cpt202g33.meetingbooking.mapper.BuildingMapper
buildingMapper;
17     @MockBean private com.cpt202g33.meetingbooking.mapper.FeedbackMapper
feedbackMapper;
18     @MockBean private com.cpt202g33.meetingbooking.mapper.RoomMapper roomMapper;
19     @MockBean private com.cpt202g33.meetingbooking.mapper.RoomTypeMapper
roomTypeMapper;
20     @MockBean private com.cpt202g33.meetingbooking.mapper.UsersMapper usersMapper;
21
22     @Test
23     void testGetBookingWithNameByBookingID() throws Exception {
24         BookingListWithNameVO vo = new BookingListWithNameVO();
25         vo.setBooking_id(123);
26         vo.setRoom_name("Conf Room");
27         // Assume Mockito is statically imported or use Mockito.when
28         when(bookingService.getBookingWithNameByBookingID(123)).thenReturn(vo);
29
30         mockMvc.perform(get("/admin/bookings/{bookingId}", 123) // Standard MockMvc
usage
31             .session(adminSession()) // Assuming adminSession() is defined elsewhere
32             .andExpect(status().isOk())
33             .andExpect(jsonPath("$.code").value(200)) // Assuming expected value 200
34             .andExpect(jsonPath("$.data.booking_id").value(123)) // Assuming expected value
123
35             .andExpect(jsonPath("$.data.room_name").value("Conf Room")); // Assuming
expected value "Conf Room"
36     }
37
38     // Example placeholder for the session helper method
39     private MockHttpSession adminSession() {
40         MockHttpSession session = new MockHttpSession();
41         // Setup session attributes for admin user if needed
42         // e.g., session.setAttribute("userRole", "ADMIN");

```

```

43     return session;
44 }
45 }
46

```

Listing 3: Unit Test Case 3 for Get Booking Record by Booking ID

```

1  @SpringBootTest
2  @Transactional // Automatic rollback, avoid pollute the database
3  @ActiveProfiles("test")
4  class AdminUserServiceImplIT { // Assuming 'IT' suffix for Integration Test
5
6      @Autowired
7      private AdminUserService adminUserService; // Assumed full name
8      @Autowired
9      private UsersMapper usersMapper;
10     @MockBean
11     private EmailSender emailSender; // Mock external dependency
12
13     private Integer testUserId;
14
15     @BeforeEach
16     void setUp() {
17         // insert test user
18         Users user = new Users();
19         user.setUsername("testUser");
20         user.setPassword("12345678"); // Assuming a password was set
21         user.setEmail("testUser@example.com");
22         user.setRole("user");
23         user.setIs_locked(0); // Initial state: unlocked
24         usersMapper.insert(user);
25         // Retrieve the auto-generated ID after insertion
26         testUserId = user.getUserId();
27     }
28
29     @Test
30     void testUnlockUser_withLockedUser_shouldUnlockSuccessfully() {
31         // Arrange: Ensure the user is locked before testing unlock
32         Users user = usersMapper.selectById(testUserId);
33         user.setIs_locked(1); // Set state to locked
34         usersMapper.updateById(user); // Update database
35
36         // Act: Call the service method to unlock the user
37         adminUserService.unlockUser(testUserId);
38
39         // Assert: Verify the user is now unlocked
40         Users updatedUser = usersMapper.selectById(testUserId);
41         assertEquals(0, updatedUser.getIs_locked()); // Expect 0 (unlocked)
42     }
43 }
44

```

Listing 4: Integration Test Case for Lock/Unlock Users

Test Results Table

Table 1: Types of testing and Techniques used

Test Type	Tools	Key Technologies
Mail address validation unit test	JUnit 5	Call static methods directly without dependency isolation
User list Service test	Mockito + JUnit 5	Simulate database mapper to verify business logic and data conversion
Booking enquiry Controller test	MockMvc + Spring Test	Simulate HTTP request, verify JSON response structure and status code
User Unlocking Integration Test	Spring Test + H2	Use H2 database, automatic transaction rollback, simulation of user state changes
Acceptance test	User testing	Simulate the real user operation process to verify the end-to-end functionality and user experience

Table 2: Typical Test Data and Results

Test Case Name	Input Data	Purpose	Expected Result	Actual Result	Pass/Fail
Verify Valid Emails	user@example.com ...	To test valid email formats	All emails return true	All emails returned true	Pass
Verify Invalid Emails	null, "", "plain address", "user@invalid..com", ...	To test invalid email formats	All emails return false	An illegal email address passed	Fail & Fixed
Get User List (Service Layer)	Mocked user: { user_id: 123, username: "testUser", avatar_url: "avatar/test.png" }	To verify correct user list retrieval	List size = 1; user_id = 123; username = "testUser"; avatar_url contains "cdn.qiniu.com"	Same as expected	Pass
Get Booking by ID (Controller Layer)	Booking ID: 123	To verify booking retrieval by booking ID	HTTP 200; JSON body contains booking_id=123 and room_name="Conf Room"	Same as expected	Pass
Unlock Locked User (Integration Test)	Test user inserted: { user_id: 1, username: "lockedUser", status: "locked" }	To verify unlock functionality	User status updated to unlocked in DB	User status updated successfully	Pass
Acceptance Testing	User's operation	Verify the System meets user's needs	Function meets the spec-sheet, the operation meets user's habits	Same as expected	Pass

Table 3: Service Layer Test Coverage

Service Implementation Class	Class (%)	Methods (%)
AdminUserServiceImpl	100 (1/1)	87 (7/8)
BookingServiceImpl	100 (1/1)	36 (4/11)
FeedbackServiceImpl	100 (1/1)	33 (4/12)
QiniuServiceImpl	100 (1/1)	71 (5/7)
RoomServiceImpl	100 (2/2)	18 (3/16)
UsersServiceImpl	100 (1/1)	44 (8/18)

Use Case Diagram

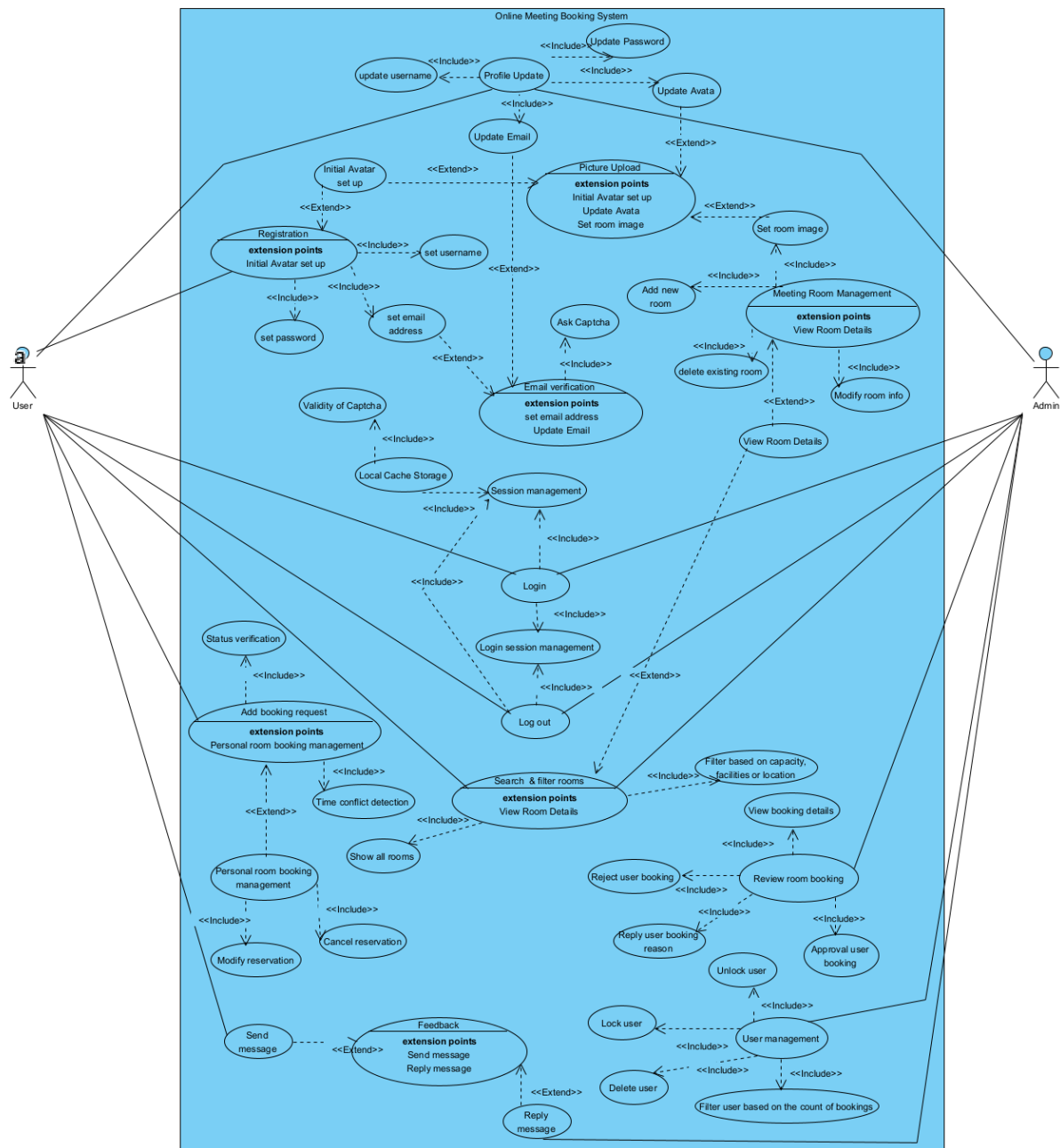


Figure 10: use Case Diagram

Project Component Diagram

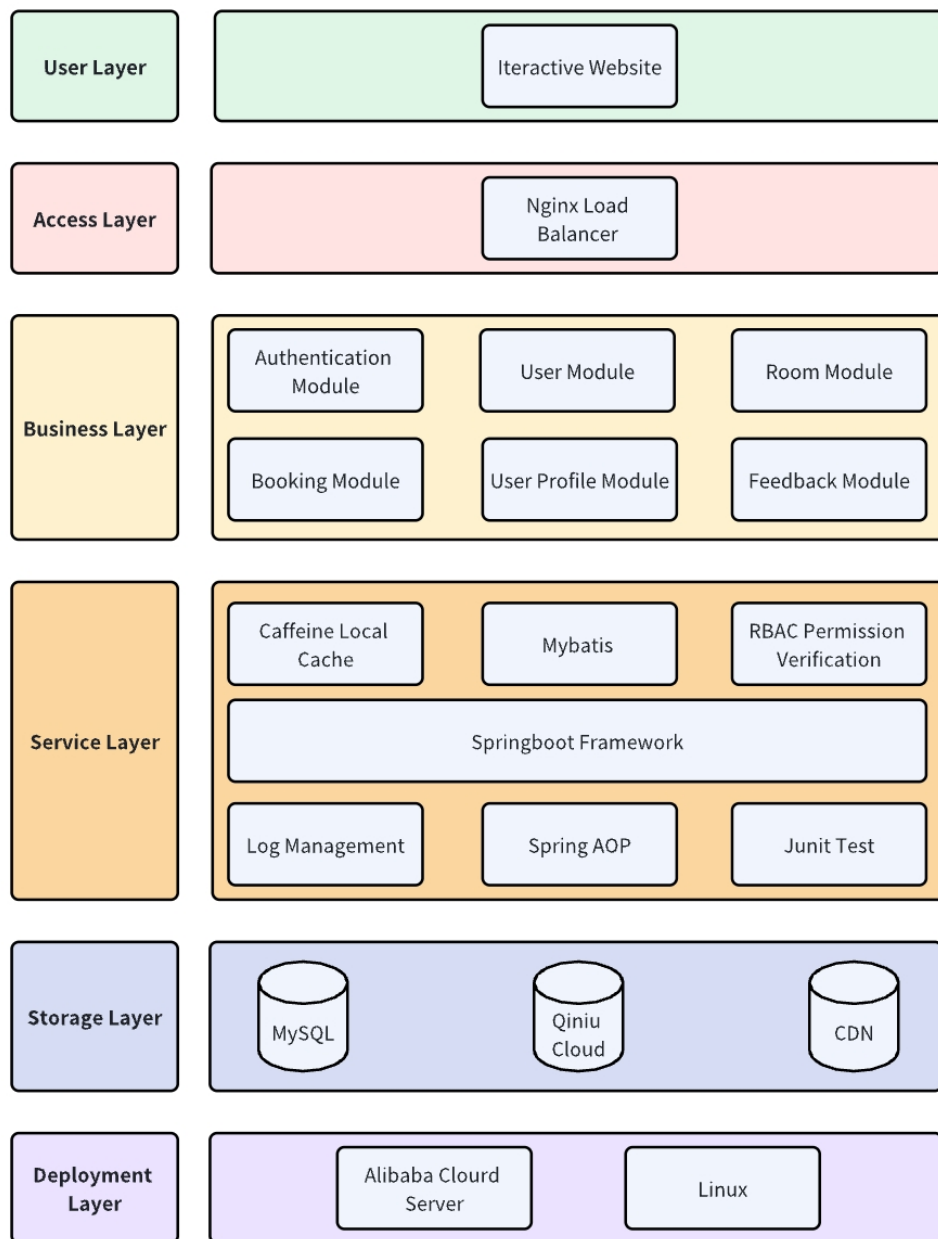


Figure 11: Project Component Composition

Sprint Diagrams

Sprint I					
Sprint Duration	Start Date: 3 rd March 2025 End Date: 14 th March 2025				
Sprint Goal	1. Set up Github repository and project structure; 2. Design database; 3. Design backend framework; 4. Design website UI prototype.				
	Mon	Tue	Wed	Thu	Fri
Week 3	Set up Github repository and project structure		Daily Review	Design backend framework	
				Design website UI prototype	
	Design database				
Week 4	Design backend framework				Daily Review
	Design website UI prototype				
	Design database				
Task Distribution	Task 1: Daiyan Wu Task 2: Yichuan Zhang, Lekun Liu, Xuning Hu Task 3: Junhao Huang, Ruiyang Task 4: Yiyang Han, Tiantian Ye, Weizhe Sun				

Figure 12: Sprint 1

Sprint II					
Task Distribution	Start Date: 17 th March 2025 End Date: 28 th March 2025				
Task Distribution	1. Set up database schema; 2. Implement backend framework and API structure; 3. Develop reusable frontend UI components.				
	Mon	Tue	Wed	Thu	Fri
Week 5	Set up database schema		Daily Review	Set up database schema	
	Implement backend framework and APIs			Implement backend framework and APIs	
	Develop reusable frontend UI components			Develop reusable frontend UI components	
Week 6	Set up database schema		Daily Review		
	Implement backend framework and APIs				
	Develop reusable frontend UI components				
Task Distribution	Task 1: Yichuan Zhang, Xuning Hu Task 2: Junhao Huang, Ruiyang Wu, Lekun Liu, Daiyan Wu Task 3: Yiyang Han, Tiantian Ye, Weizhe Sun				

Figure 13: Sprint 2

Sprint III					
Sprint Duration		Start Date: 31 st March 2025 End Date: 18 th April 2025			
Sprint Goal		1. Implement core backend logic and connect with database; (Junhao Huang, Ruiyang Wu, Lekun Liu, Daiyan Wu, Yichuan Zhang, Xuning Hu) 2. Develop frontend logic and integrate with backend APIs; (Yiyang Han, Tiantian Ye, Weizhe Sun, Lekun Liu) 3. Set up OSS Object Storage Service. (Yichuan Zhang)			
	Mon	Tue	Wed	Thu	Fri
Week 7	Implement core backend logic and connect with database		Daily Review	Implement core backend logic and connect with database	
	Develop frontend logic and integrate with backend APIs			Develop frontend logic and integrate with backend APIs	
	Set up OSS Object Storage Service			Set up OSS Object Storage Service	
Week 8	Implement core backend logic and connect with database		Daily Review	Implement core backend logic and connect with database	
	Develop frontend logic and integrate with backend APIs			Develop frontend logic and integrate with backend APIs	
	Set up OSS Object Storage Service			Set up OSS Object Storage Service	
Week 9	Implement core backend logic and connect with database		Daily Review	Implement core backend logic and connect with database	
	Develop frontend logic and integrate with backend APIs			Develop frontend logic and integrate with backend APIs	
	Set up OSS Object Storage Service			Set up OSS Object Storage Service	
Task Distribution		Task 1: Junhao Huang, Ruiyang Wu, Lekun Liu, Daiyan Wu, Yichuan Zhang, Xuning Hu Task 2: Yiyang Han, Tiantian Ye, Weizhe Sun, Lekun Liu Task 3: Yichuan Zhang			

Figure 14: Sprint 3

Sprint IV					
Sprint Duration		Start Date: 21 st April 2025 End Date: 30 th April 2025			
Sprint Goal		1. Deploy backend & frontend onto the server; 2. Conduct backend functional testing & Fix bugs; 3. Perform UI/UX testing on frontend & Fix display and logic issues; 4. Verify production environment functionality.			
	Mon	Tue	Wed	Thu	Fri
Week 10	Daily Review	Deploy backend & frontend onto the server	Conduct backend functional testing & Fix bugs		
			Perform UI/UX testing on frontend & Fix display and logic issues		
Week 11	Conduct backend functional testing & Fix bugs	Verify production environment functionality	Deadline		
	Perform UI/UX testing on frontend & Fix display and logic issues				
Task Distribution		Task 1: Lekun Liu, Daiyan Wu, Yichuan Zhang, Task 2: Junhao Huang, Ruiyang Wu, Lekun Liu, Daiyan Wu, Yichuan Zhang, Xuning Hu Task 3: Yiyang Han, Tiantian Ye, Weizhe Sun Task 4: All team members			

Figure 15: Sprint 4

Acceptance Test Screenshots

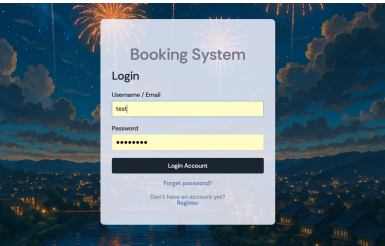


Figure 16: Screenshot of user login

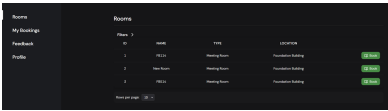


Figure 17: Screenshot of room list

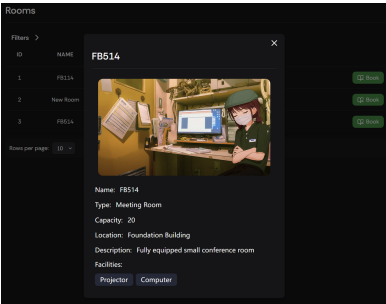


Figure 18: Screenshot of room detail

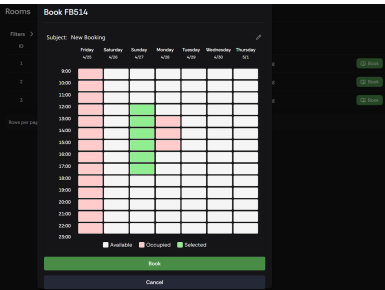


Figure 19: Screenshot of booking time table

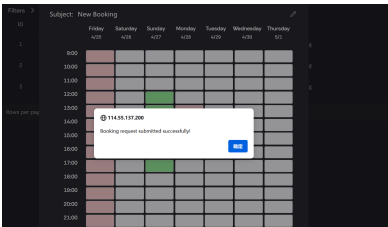


Figure 20: Screenshot of success reply

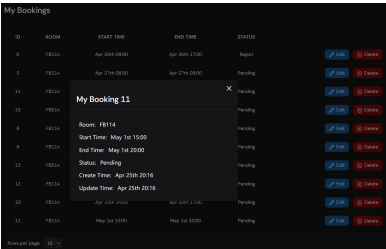


Figure 21: Screenshot of booking history